AutoVet System Architecture

ORM, routing, packages, validation, roles, authentication and duplicate prevention, explained in simple terms

1. ORM: Eloquent (Laravel)

In simple terms: an ORM lets the code work with the database through objects instead of writing SQL by hand.

AutoVet uses **Eloquent**, which comes with Laravel. Each table has a "model" class, for example `Appointment`, `Pet`, `Owner` and `Inventory`. The code can write `Appointment::create([...])` or `$appointment->pet->owner`, and Eloquent turns that into SQL for the database (MySQL locally, TiDB in production).

2. Routing System (Exposed Endpoints)

Backend: Laravel router

Defined in `backend/routes/api.php`, with about **190 API endpoints**. These are the URLs the frontends call. For example:

- `GET /api/inventory` lists stock.
- `POST /api/appointments` books a visit.
- `POST /api/appointments/{id}/decline` declines an appointment.

Endpoints sit behind security layers: you must be logged in (`auth:sanctum`), and some groups also require a certain role (`role:...`).

Frontend: React Router v7

Both the admin app and the client portal use **React Router** (`createBrowserRouter`). It maps page URLs such as `/appointments` and `/inventory` to screens without reloading the page.

3. Routing History and Query Params

History: React Router uses the browser's history, so the Back and Forward buttons move between pages like on a normal website, even though it is a single-page app.

Query params are the `?key=value` part of a URL. They pass small pieces of information to a page:

Example	What it does
`/appointments?patientId=12`	The "Book appointment" button on a patient's profile opens Appointments with that pet already selected.
`/settings?tab=reviews`	Opens Settings straight to a specific tab.
`/reset-password?token=…&email=…`	The password-reset link in the email carries the user's reset token.
`/api/invoices?appointment_id=5&per_page=1`	The frontend sends query params to the server for filtering and paging.

4. Main Packages

Part	Package	What it does
Backend framework	Laravel 12 (PHP 8.2)	The base of the whole server
Login tokens	Laravel Sanctum	Issues and checks login tokens
Real-time	Laravel Reverb / Pusher + Laravel Echo	Live updates without refreshing
PDF	DomPDF (server), jsPDF (browser)	Invoices and reports
Email	Brevo mailer	Sends notification emails
File storage	Flysystem S3	Stores uploaded images
Phone numbers	libphonenumber	Checks phone number formats
Error tracking	Sentry	Records server errors
Frontend	React 19, Vite, TypeScript (portal)	Builds the two web apps
Data loading	TanStack React Query	Fetches and caches API data
Forms	React Hook Form + Zod	Form handling and validation
Forecasting	scikit-learn (Python)	The linear regression forecasts

5. Database Validator

AutoVet checks data at three levels, so bad data is stopped as early as possible:

1. In the browser (Zod): checks forms before they are sent, for example a required field or a valid email.
2. On the server (Laravel Validator): every request is checked again with `$request->validate([...])`.
Some rules look at the database itself:

- `exists:pets,id` : the pet must actually exist.
- `unique:admins` : the email must not already be taken.
- Others check format: `required`, `date`, `date_format:H:i`, `min:10` and so on.

3. In the database itself: foreign keys stop broken links. For example, a pet cannot point to an owner that does not exist.

6. Roles and Their Purpose

Role	Who	What they can do
Super Admin	System owner	Manages all clinics, announcements and system logs
Clinic Admin	Clinic manager	Runs the clinic: settings, users, inventory, finances
Veterinarian	Doctor	Clinical work: diagnoses, medical records, AI Clinical Support, analytics
Staff	Front desk	Appointments, dashboard, daily operations; no financial reports
Owner	Pet owner (portal)	Books visits, manages their pets, views their invoices

In the code, roles are grouped: **admin roles** (Super Admin, Clinic Admin, Vet) can make changes; **dashboard roles** add Staff for reading the dashboard; **clinical roles** (Vet, Clinic Admin) can write medical records.

7. Authentication Stack

- **Laravel Sanctum token login.** After logging in, the user gets a token that the app sends with every request.
- **Two separate account types.** Staff log in through the `admins` table and pet owners through `portal_users`. Each has its own Sanctum guard, so a portal account cannot reach admin endpoints.
- **Google sign-in** for the portal. The server checks the Google login with Google, then issues its own Sanctum token (valid for 30 days).
- **Role checks.** Middleware (`CheckRole` / `RoleMiddleware`) blocks requests from the wrong role.

8. Unique IDs and Duplicate Prevention

Method	How it prevents duplicates
UUID on every record	Besides its normal ID number, each record gets a random unique code such as `3f2a9c1e-…` (`HasSyncFields`). It lets records be matched correctly when the local and live databases sync, so the same record is not created twice.
Google ID	A Google account is matched by its permanent Google ID (`google_id`) or email, so signing in with Google twice does not create two accounts.
Unique email	An email cannot be registered twice. This is checked across both the staff and portal account tables.
Walk-in checks	An owner's phone number cannot be registered twice, and the same owner cannot have two pets with the same name.
Booking checks	The same pet or vet cannot be double-booked at one time, and a time slot holds at most 2 approved appointments.